

Task-Centric, Checkpoint-Gated Verification for Automated Software Hardening

An engineering case study, not a claim of novel research contribution. The technique composes existing ideas (adversarial multi-agent critique, checkpoint-based retry, evidence-scored findings) into a working pipeline for a specific applied problem: finding and fixing real defects in a large existing codebase without a human reviewing every step. What follows is a report on what was built, what it replaced, and the evidence that the replacement was actually better — not a paper claiming the underlying ideas are new.

1. Problem statement

Given a codebase of non-trivial size (here: `browser-use`, an async Python browser-automation library), find real, reproducible defects and fix them, using LLM agents as the labor, with two hard constraints:

1. **No human reviews every step.** A supervising process (here, an orchestrating Claude Code session) sets policy and does final review, but individual claims, reproductions, and implementations must be judged by other agents, not a human in the loop at every stage.
2. **Findings must be evidence-based, not self-reported.** An agent's claim that it fixed something is not evidence that it did.

2. Iteration 1: self-critique-in-one-call (rejected)

The first working design had one agent per target file: read the file, find a defect, argue against its own finding ("is this actually wrong, or intentional?"), then implement and test if it survived its own critique.

This is a known-weak pattern and the weakness showed up exactly where the literature predicts: a model grading its own output shares the same blind spots that produced the output in the first place. Concretely, in this effort:

- Two real defects were only caught by an *external* re-verification pass (a different context, not the agent that wrote the fix) — never by the agent's own inline self-critique.
- One fix accepted by self-critique was later shown, under adversarial review, to be worse than the bug it targeted (an uncapped data structure that could overflow a downstream size budget) — the self-critique step did not catch it because the failure mode was outside what the same context had already reasoned about.

This matches the standard critique of self-consistency-only approaches: self-refinement without an external signal converges on the model's own prior, not on ground truth. It also matches the core argument for constitutional/critique-based methods generally (Bai et al., *Constitutional AI: Harmlessness from AI Feedback*, Anthropic, 2022) and for structured self-feedback loops (Madaan et al., *Self-Refine: Iterative*

Refinement with Self-Feedback, NeurIPS 2023) — both establish that self-feedback helps, but neither claims it substitutes for an independent check when the check and the claim share a context window.

3. Iteration 2: task-centric pipeline with mandatory checkpoint-evaluator gates

The replacement design, in the vocabulary of the pipeline:

HUNT -->	CHECKPOINT/EVALUATE -->	REFUTE -->	CHECKPOINT/EVALUATE -->	IMPLEMENT -->	VERIFY
(claim	(pass/weak/poor:	(a DIFFERENT	(same gate)	(survivors	(mechanical,
only,	continue /	model tries		only)	no model
no code,	retry same	to falsify			judgment
cheap	model /	the claim)			needed)
model)	reassign				
	different				
	model)				

- **HUNT** produces a *claim*, not a patch — a specific, falsifiable assertion with file:line evidence, before any code is written. This separates hypothesis generation from hypothesis testing, the same separation that makes null-hypothesis testing meaningful in any empirical discipline.
- **The evaluator checkpoint is mandatory, not advisory.** Every hunt claim and every refute verdict is judged by a separate agent call before the pipeline proceeds. A `pass` continues; a `weak` verdict retries the same model with the evaluator's specific correction appended; a `poor` verdict retires the attempt and reassigns the same task, from the same checkpoint state, to a *different model*. This is the checkpoint-gated reassignment mechanism, and it is deliberately not a live interrupt of a running inference call (no such primitive exists in the tool used here, or in any known LLM-agent framework as of this writing) — it is a between-call decision made at a state boundary that already exists.
- **REFUTE is adversarial by construction, not by instruction.** Rather than asking one agent to "be critical of itself" (iteration 1's failure mode), a *structurally different* agent — different call, different context, and in the final round, a different model — is given the explicit goal of falsifying the claim: reproduce it failing on unmodified code, or show it is already-intended behavior. This is closer to Popperian falsificationism applied to bug reports than to any specific ML technique, and it is also the applied shape of adversarial multi-agent debate (Du, Li, Torralba, Tenenbaum, and Mordatch, *Improving Factuality and Reasoning in Language Models through Multiagent Debate*, 2023): independent agents reach a more reliable joint verdict than one agent polling itself.
- **Model diversity is used as an ensemble-diversity signal, not a cost optimization.** The hunting model and the refuting model were deliberately different tiers in the final round (a cheaper model hunts, a stronger model refutes and implements) specifically so that the two stages do not share failure modes correlated with one model's training.
- **VERIFY is mechanical wherever possible.** Whether a new test fails before a fix and passes after, whether lint/type-checking are clean, whether the existing suite still passes — none of this needs model judgment, and treating it as a model-judged step wastes the tier's reasoning budget on

something a shell command answers exactly.

4. Evidence that iteration 2 outperformed iteration 1

Not a controlled experiment — a single natural before/after inside one effort, reported as such. In the round run under iteration 2's design, the refute stage did substantive, non-rubber-stamp work on every one of its three targets:

1. It corrected a hunter's predicted failure *mechanism* (a query engine's actual degrade-on-malformed-input behavior, discovered empirically, rather than the exception-raising behavior the hunter assumed) while confirming the hunter's conclusion was still correct.
2. It discovered the hunted bug was *more severe* than described (an actual focus-state hijack, not merely a misleading log line) and simultaneously *downgraded* the finding's practical severity score, because the same investigation revealed the affected code path is not currently reachable in production (dead code — never instantiated).
3. It found the identical defect class in two additional, unclaimed code paths (two other LLM providers with the same asymmetry) while correctly declining to expand the current claim's scope to fix them, flagging them instead as separate future work.

None of these three corrections occurred in the self-critique-in-one-call rounds that preceded this one, despite comparable per-target reasoning budget. The full evidence trail (per-agent tokens, verdicts, and outcomes) is in `docs/agent-notes/performance-registry.json` and `docs/agent-notes/model-performance.md`; the fixes themselves are in `docs/agent-notes/ledger.json`.

5. What is not established

- **Sample size is small** (one round, three targets) — this is a case study, not a benchmark. A rigorous claim would need many rounds across many codebases with a held-out defect set for recall measurement.
- **No controlled ablation.** The two iterations differed in more than one variable (self-critique vs. external critique, and single-model vs. model-diverse), so the evidence above cannot isolate which change did the work. A follow-up that holds model choice fixed and varies only the critique structure (or vice versa) would be needed to say more.
- **The evaluator itself is unverified.** Nothing here checks whether the evaluator's own pass/weak/poor judgments are well-calibrated; it is possible for a systematically miscalibrated evaluator to look identical to a well-calibrated one in a small sample where every gate happened to pass.

6. Reproducing this

`docs/agent-notes/workflows/round3-task-centric-pipeline.js` is the literal script. It is written against a specific in-house orchestration tool (the `Workflow` primitive described in `docs/agent-notes/README.md`, section A); porting the *pattern* — claim-then-falsify-then-implement, with a mandatory external gate at each transition — to a different agent framework requires only that framework's equivalent of "run one agent call to completion and inspect its structured output before deciding the next call," which is a close-to-universal primitive in current LLM tool-use frameworks.